Implementing DreamWaQ Paper

Robust Quadrupedal Locomotion with Implicit Terrain Imagination via Deep Reinforcement Learning

A Simulation Reproduction in NVIDIA Isaac Lab

Roman Kalyna



Table of Contents

Table of Contents 2
Abstract 3
1. Introduction 3
1.1 Background and Motivation 3
1.2 The DreamWaQ Paper 3
1.3 Project Goals and Scope 4
2. Method: The DreamWaQ Approach 4
2.1 Asymmetric Actor-Critic 4
2.2 CENet: Context-aided Estimator Network 4
2.3 Training Algorithm (PPO) and Domain Randomization 5
3. Implementation 6
3.1 Simulation Platform and Robot 6
3.2 Asymmetric Observations 6
3.3 CENet Implementation and Integration 7
3.4 Key Design Decisions 7
3.5 Verification Strategy 7
3.6 Program Structure and Main Functions 8
4. Experiments and Results 9
4.1 Experimental Setup 9
4.2 Learning Curves 10
4.3 Command Tracking 12
4.4 CENet Velocity Estimation 14
4.5 Qualitative Results: Trained Robot in Simulation 15
5. Discussion 16
5.1 Comparison with the Paper 16
5.2 Limitations 17
5.3 Future Work 17
6. Conclusion 18
References 18
Appendix A: Code and Reproduction 19
Appendix B: Running the Full Pipeline and Deploying the Policy 19

Abstract

This project reproduces the core of DreamWaQ (Nahrendra, Yu, and Myung, ICRA 2023), a single-phase deep-RL framework for rough-terrain quadrupedal locomotion that deploys on proprioception alone. Two of its three contributions were implemented in NVIDIA Isaac Lab on a Unitree Go2: the asymmetric actor-critic (the actor sees only proprioception; the critic also sees the privileged terrain height scan and the true base velocity) and the CENet estimator, whose inferred body velocity and latent context feed the actor. Trained for 2000 iterations against a privileged symmetric baseline, DreamWaQ matched it on terrain difficulty and stability and trailed only slightly on reward and tracking, while CENet estimated the body velocity to an MSE of about 0.006 $(m/s)^2$ in evaluation rollouts, giving near-baseline locomotion from deployable observations alone.

1. Introduction

1.1 Background and Motivation

Legged robots can cross terrain that wheeled and tracked platforms cannot, but controlling them over uneven ground is hard: a quadruped must coordinate twelve or more joints while reacting to disturbances, varying friction, and terrain it cannot see in advance. Classical model-based controllers need accurate models and hand-tuning, which limits their robustness in messy real-world settings.

Deep reinforcement learning (RL) takes a different route: train the policy across many simulations running at once, then deploy it on hardware. The difficulty is the gap between what the robot can use during training and after deployment. In simulation the learner has privileged information, the exact terrain under the robot and its true body velocity, but a real robot has only proprioceptive sensors (joint encoders and an IMU) and cannot measure these directly, so a policy that relies on them cannot be deployed. DreamWaQ's idea, which this project follows, is to let the robot infer that hidden context from its own recent proprioceptive history.

1.2 The DreamWaQ Paper

DreamWaQ (Nahrendra, Yu, and Myung, ICRA 2023) is a deep-RL framework for robust quadrupedal locomotion. Its main idea is "implicit terrain imagination": rather than giving the deployed policy a direct terrain map, DreamWaQ trains a context estimator that infers a hidden representation of the robot's state and surroundings from proprioceptive history alone. The key feature of the method is that it trains in a single phase. Many earlier approaches use a two-stage teacher-student scheme, in which a privileged "teacher" policy is trained first and a deployable "student" is then trained to imitate it. DreamWaQ instead trains the policy, the value function, and its context estimator together, end-to-end, which keeps the pipeline simple. The paper lists three contributions: the asymmetric actor-critic, the Context-aided Estimator Network (CENet), and a real-world robustness and durability evaluation on a physical robot. The method also introduces an adaptive bootstrapping mechanism (AdaBoot) during training. This project reproduces the first two contributions; the real-world evaluation is outside the scope of a simulation study, and AdaBoot was not implemented (see Section 5.2).

1.3 Project Goals and Scope

- Reproduce the paper's two implemented core contributions (the asymmetric actor-critic and CENet) in NVIDIA Isaac Lab.
- Train and compare a baseline policy against the DreamWaQ policy.
- Analyze the Context-aided Estimator Network (CENet) and visualize the trained robot.

2. Method: The DreamWaQ Approach

2.1 Asymmetric Actor-Critic

Training uses two networks: the actor (the deployed policy) and the critic, a training-only aid that scores each state so the actor learns faster, discarded after training. "Asymmetric" means they receive different inputs. The actor sees only what real hardware can sense (proprioception: base angular velocity, projected gravity, the velocity command, joint positions and velocities, and the last action) plus CENet's estimated velocity and latent context while the critic also sees privileged, simulation-only information: a terrain height scan and the true base linear velocity. The better-informed critic gives the actor a cleaner learning signal; the actor, never seeing privileged data, cannot depend on it, yielding a proprioception-only policy that is deployable as-is and trained in a single phase with no teacher network.

2.2 CENet: Context-aided Estimator Network

CENet is how the actor recovers the information it is not allowed to observe directly. It takes a short history of proprioceptive observations, passes it through a shared encoder, and splits into two output heads, one for velocity and one for context.

The first head estimates the robot's true body linear velocity. It is trained by comparing its estimate against the ground-truth velocity available in simulation, using a mean-squared-error (MSE) loss.

The second head is a β-variational autoencoder (β-VAE), which produces the context: it compresses the observation history into a small latent vector z and is trained to reconstruct the observation from z, so z must capture what matters. A Kullback-Leibler (KL) term keeps z close to a standard-normal prior, and the weight β sets how firmly that rule is enforced: strong enough for a smooth, well-behaved latent that generalizes, but weak enough that z still carries information.

Because both heads share one encoder, it must compress the history into a representation useful for both jobs at once. The resulting velocity estimate and latent context are then handed to the actor as a deployable stand-in for the privileged information it cannot measure. CENet is trained with a single combined (hybrid) loss:

$$L = \mathrm{MSE}(\hat{v}, v) + \mathrm{MSE}(\hat{o}, o) + \beta \cdot \mathrm{KL}(z \,\|\, N(0, I))$$

where $\hat{v}$ is the estimated body velocity, v is the ground-truth velocity, $\hat{o}$ is the reconstructed observation, o is the target observation, z is the latent context, and β controls the strength of the KL regularization. The first term trains the velocity estimator, the second trains the autoencoder reconstruction, and the third keeps the latent space well-structured.

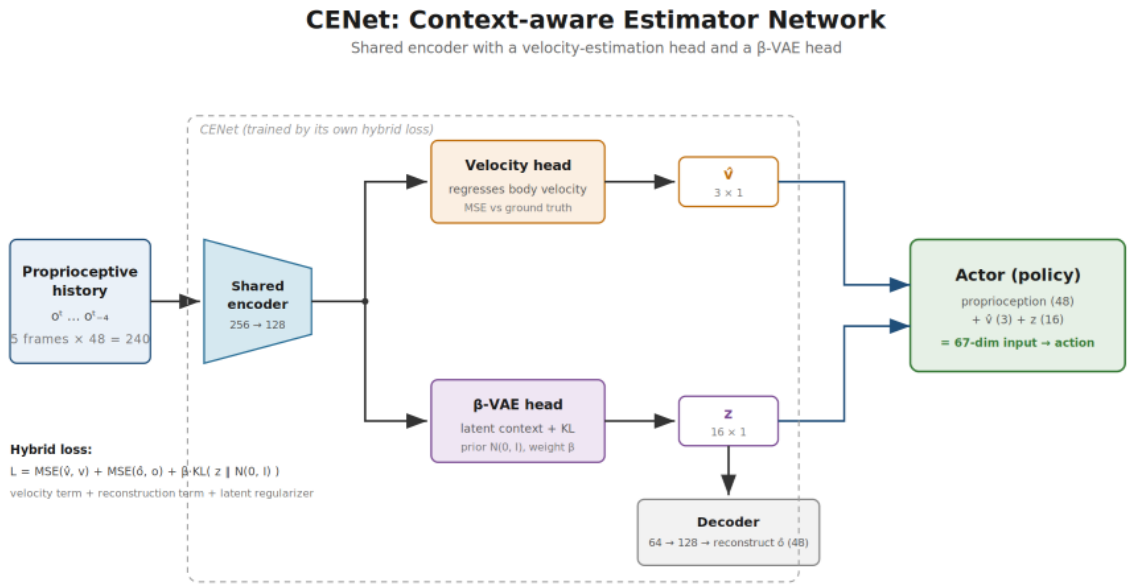


Figure 1: CENet architecture and how its outputs feed the policy.

2.3 Training Algorithm (PPO) and Domain Randomization

The policy is trained with Proximal Policy Optimization (PPO), a standard on-policy algorithm that improves the policy in small, clipped steps for stable training; the settings follow the paper (table below). For robustness and sim-to-real transfer, each environment's physical parameters are domain-randomized (payload, Kp/Kd actuator gains, motor strength, center of mass, ground friction, control delay), and a terrain curriculum advances the policy from easy to harder terrain as it improves.

Table 1: PPO settings (parameters)

PPO setting	Value
Clipping parameter	0.2
GAE factor λ	0.95
Discount factor γ	0.99
Optimizer / learning rate	Adam, 1×10^{-3} (adaptive-KL schedule)
Activations	ELU
Parallel environments	2048 (paper: 4096; limited by 8 GB GPU)

3. Implementation

3.1 Simulation Platform and Robot

The method was implemented in NVIDIA Isaac Lab (version 2.0, running on Isaac Sim 4.5), a GPU-accelerated robot-learning framework, using the rsl_rl library (version 3.1.2) for the PPO implementation. The robot is the Unitree Go2, a small quadruped with twelve actuated joints. Training ran on a single NVIDIA RTX 4060 Laptop GPU with 8 GB of memory, simulating 2048 environments in parallel at roughly 11,000-13,000 environment steps per second. Rather than editing the Go2 velocity-tracking task supplied with Isaac Lab, the implementation takes it into a new, self-contained task package and subclasses the configuration. That keeps the original task intact as a known-good reference and isolates all DreamWaQ-specific changes. The network and training design follow the DreamWaQ paper and a public reference implementation that targets Isaac Gym.

Table 2: Implementation environment.

Component	Value
Simulator	NVIDIA Isaac Lab 2.0 (Isaac Sim 4.5)
RL library	rsl_rl 3.1.2 (PPO)
Robot	Unitree Go2 (12 actuated joints)
GPU	NVIDIA RTX 4060 Laptop, 8 GB VRAM
Parallel envs	2048
Task base	Isaac-Velocity-Rough-Unitree-Go2 (forked)

3.2 Asymmetric Observations

In the stock task, proprioception and the terrain scan sit in one group seen by both networks. To implement the asymmetry they were split into separate groups, and rsl_rl 3.1.2's obs_groups routing maps groups to the actor and critic independently, so the asymmetry is achieved in configuration, with no library changes. The architectures confirm it: actor input 64 (45 proprioception + CENet's 3 velocity + 16 latent), critic input 235 (45 proprioception + 187 height scan + 3 true base velocity).

Table 3: Observation groups (dimensions and contents)

Group	Dim	Contents	Seen by
policy	45	ang. velocity, gravity, command, joint pos/vel, previous action (base linear velocity excluded: unmeasurable on hardware, CENet's job)	Actor + critic
height_scan	187	17×11 grid of terrain heights (privileged)	Critic only

privileged_vel	3	true base linear velocity	CENet target + critic

3.3 CENet Implementation and Integration

CENet was implemented as a self-contained PyTorch module. Its input is a flattened history of the last five proprioceptive frames (5 × 45 = 225 dimensions). A shared encoder (hidden layers 256 and 128) feeds two heads. The velocity head outputs a 3-dimensional body-velocity estimate. The β-VAE head produces a 16-dimensional latent context (using a mean and log-variance, sampled via the reparameterization trick).

CENet was integrated into the training loop through two subclasses. A custom actor-critic subclass creates CENet, runs it on the proprioceptive-history observation group, and joins the resulting velocity estimate and latent context onto the actor's input, growing the actor's input from 45 to 64 dimensions (45 proprioceptive + 3 velocity + 16 latent). A custom PPO subclass adds CENet's hybrid loss to the existing PPO objective before the backward pass, so the policy, value function, and CENet are trained together in a single update step, matching the paper's single-phase training of all three networks together. The three CENet loss components (velocity MSE, reconstruction MSE, and KL) are logged each iteration so we can watch them settle during training.

3.4 Key Design Decisions

- **Detached estimator:** CENet's outputs (the velocity estimate and latent context) are passed to the actor as fixed inputs, so the policy's learning cannot reshape CENet; CENet learns only from its own prediction error. This matches the paper's design of training the policy, value, and CENet as separate objectives in a single phase.
- **Reconstruction target:** reconstruct the current observation o_t rather than the paper's next observation o_{t+1}, to avoid modifying the rollout storage. A noted change; keeps the latent's meaning.
- **Observation history H=5:** implemented using Isaac Lab's flattened observation history.
- **Asymmetry implemented at the configuration level (observation groups),** requiring no changes to the RL library's network code.

3.5 Verification Strategy

Each component was verified with a pass/fail check before the next was built: (1) the stock task trains; (2) the forked task trains identically, confirming the package works; (3) the printed input dimensions confirm the asymmetry (actor 64, critic 235); (4) offline unit tests confirm CENet's output shapes, a finite differentiable loss, and that gradients reach the shared encoder from CENet's loss but not the actor's (the detach design); (5) a live run shows CENet losses decreasing.

3.6 Program Structure and Main Functions

The implementation is a self-contained Isaac Lab task package, go2_dreamwaq, that subclasses the stock Go2 velocity task rather than editing it. Table 4 lists the program files, table 5 summarizes the main classes and the script.

Table 4: Program files in the go2_dreamwaq package.

File	Role
__init__.py	Registers the Gym task IDs: the training task and its -Play- evaluation variant.
rough_env_cfg.py	Defines the environment and splits the observations into the policy, height_scan, and privileged_vel groups.
rsl_rl_ppo_cfg.py	Configures PPO, sets the obs_groups routing, and points the runner at the custom actor-critic and PPO classes.
cenet.py	Defines the CENet network module.
dreamwaq_actor_critic.py	Defines DreamWaQActorCritic, which wraps CENet around the stock policy.
dreamwaq_ppo.py	Defines DreamWaQPPO, the single-phase training update.
test_cenet.py, test_dreamwaq_actor_critic.py	Offline unit tests for tensor shapes, the hybrid loss, and the network input dimensions.
plot_results.py, rollout_eval.py	Generate the report figures (training curves and evaluation rollouts).

Table 5: Main classes and functions, and the script.

Component (file)	What it does
CENet (cenet.py)	forward() encodes the five-frame proprioceptive history and returns the 3-D velocity estimate and 16-D latent context; loss() returns the hybrid loss (velocity MSE + reconstruction MSE + β·KL).
DreamWaQActorCritic (dreamwaq_actor_critic.py)	Runs CENet on the history group, detaches its outputs, and joins them onto the 45-D proprioception to form the 64-D actor input; the critic input is 235-D (45 proprioception + 187 height scan + 3 true base velocity).
DreamWaQPPO (dreamwaq_ppo.py)	Adds CENet's hybrid loss to the PPO objective so the policy, value function, and CENet train together in one step; logs the three CENet loss components each iteration.
rollout_eval.py	Loads a trained checkpoint, applies a fixed velocity command, and logs commanded/actual velocity, base path, joint angles, and CENet's estimated-versus-true velocity (data for Figures 6, 7, 8, and 10).

4. Experiments and Results

4.1 Experimental Setup

To evaluate the implementation, two policies were trained and compared. The baseline is the stock Go2 velocity-tracking task: a symmetric actor-critic (both networks observing the same 235-dimensional vector, including the height scan and true base velocity) with no CENet. The DreamWaQ policy uses the asymmetric actor-critic and CENet as described above; its critic receives the same privileged information (also 235-dimensional: proprioception + height scan + true velocity), so the two critics are equally privileged and the comparison isolates the actor's observations and the addition of CENet. To keep the comparison fair, both policies were trained for the same number of iterations (2000), from the same random seed (1), with the identical terrain curriculum and reward configuration; the only differences are the asymmetric observation routing and the addition of CENet. During training, the following quantities were logged each iteration: mean episodic reward, mean episode length, the linear- and angular-velocity tracking errors, the terrain-curriculum level reached, and (for the DreamWaQ policy) the three CENet loss components.

Table 6 summarizes the final performance of both policies.

Table 6: Final training performance, baseline vs DreamWaQ (2000 iterations, seed 1).

Metric	Baseline	DreamWaQ
Mean episodic reward	20.0	17.0
Mean episode length (steps)	940	915
Linear-velocity error (m/s)	0.41	0.50
Terrain curriculum level	5.46	5.35
CENet velocity MSE (training)	-	0.012
Actor observation dim	235 (height scan + true velocity)	64 (proprio + estimates)

4.2 Learning Curves

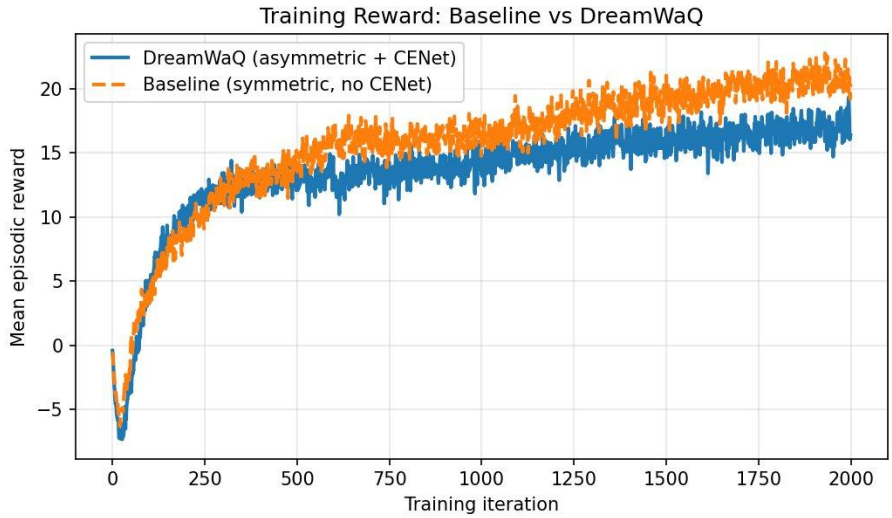


Figure 2: Training reward over 2000 iterations for the baseline (dashed) and DreamWaQ (solid) policies (seed 1, identical curriculum).

Figure 2 shows the mean episodic reward: the curves track closely for about 400 iterations, after which the baseline holds a steady lead of roughly three reward points (≈20 vs 17), the expected cost of replacing the baseline's privileged inputs with CENet's inferred state. The same pattern shows up again in the curves below.

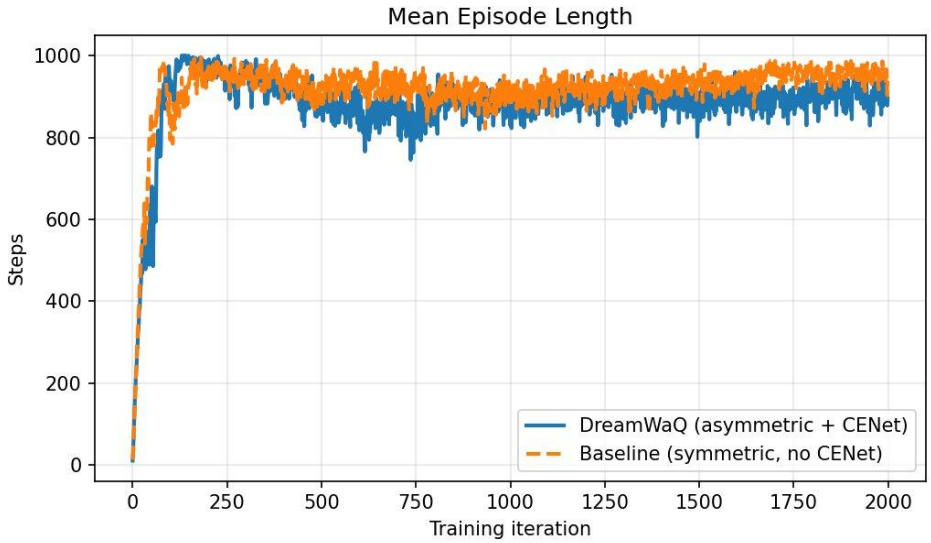


Figure 3: Mean episode length over training. Both policies settle at roughly 900-950 of 1000 possible steps (the baseline slightly higher), indicating the robots rarely fall.

Mean episode length (Figure 3) is the clearest sign of nearly equal ability: both policies rise quickly and level off in the 900-950 range out of 1000 steps, the baseline slightly higher. A high episode length means the robot avoids the fall (base-contact) termination, so both learn stable locomotion.

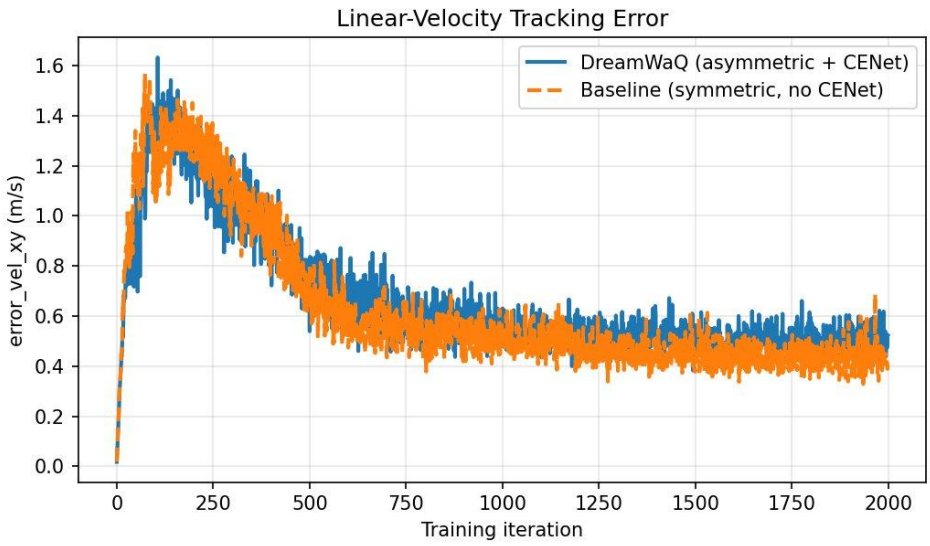


Figure 4: Linear-velocity tracking error (error_vel_xy) over training. Error spikes as the robot begins moving, then decays as the policy improves.

Figure 4 plots the linear-velocity tracking error: it spikes early as the robot starts moving, then decays as the policy matures. DreamWaQ settles slightly above the baseline (final 0.50 vs 0.41 m/s), the expected cost of tracking from inferred rather than privileged state.

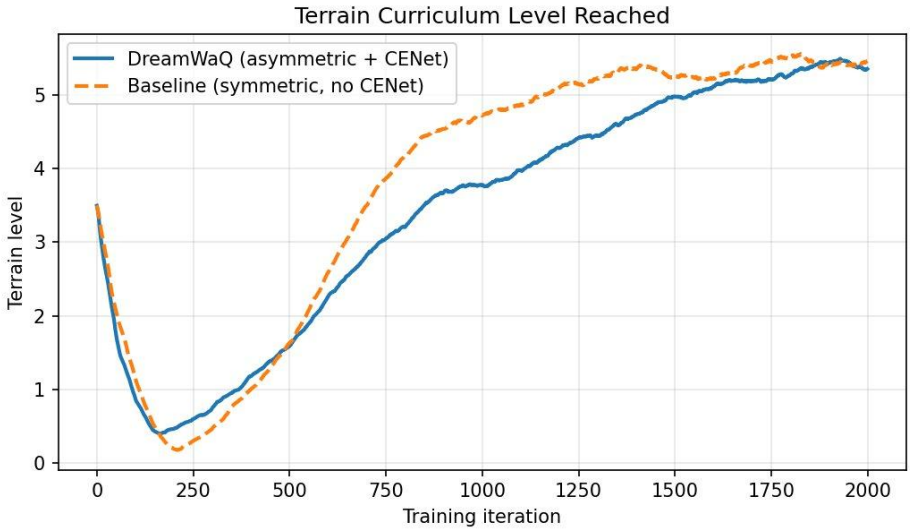


Figure 5: Terrain-curriculum level reached over training. Both policies are demoted to easy terrain early, then climb as they improve.

Figure 5 shows the terrain-curriculum level. Both policies are demoted to easy terrain (about 0.5) while learning to walk, then climb steadily. The baseline leads for most of training, but DreamWaQ catches up over the final few hundred iterations, both finishing at essentially the same level (5.46 vs 5.35), the terrain-blind policy nearly matching one that sees the terrain directly.

Summary:

On terrain difficulty and episode length the two policies finish level; DreamWaQ trails only slightly, by about three reward and roughly 0.1 m/s of tracking error. This is the expected cost of its observations: the baseline actor sees a 235-dimensional vector including the true base velocity and terrain height scan, while DreamWaQ's actor sees only 45-dimensional proprioception plus CENet's inferred velocity and latent context (64 dimensions, none privileged). It approaches the privileged baseline using only deployable observations, reproducing the paper's central claim.

4.3 Command Tracking

The trained policy was rolled out for 500 control steps (10 s) under a fixed command of 0.5 m/s forward with zero lateral velocity and yaw rate. In Figure 6 the measured velocity oscillates around the command on each axis, normal within a gait cycle with the forward channel near 0.5 m/s and the lateral and yaw channels near zero, confirming command tracking.

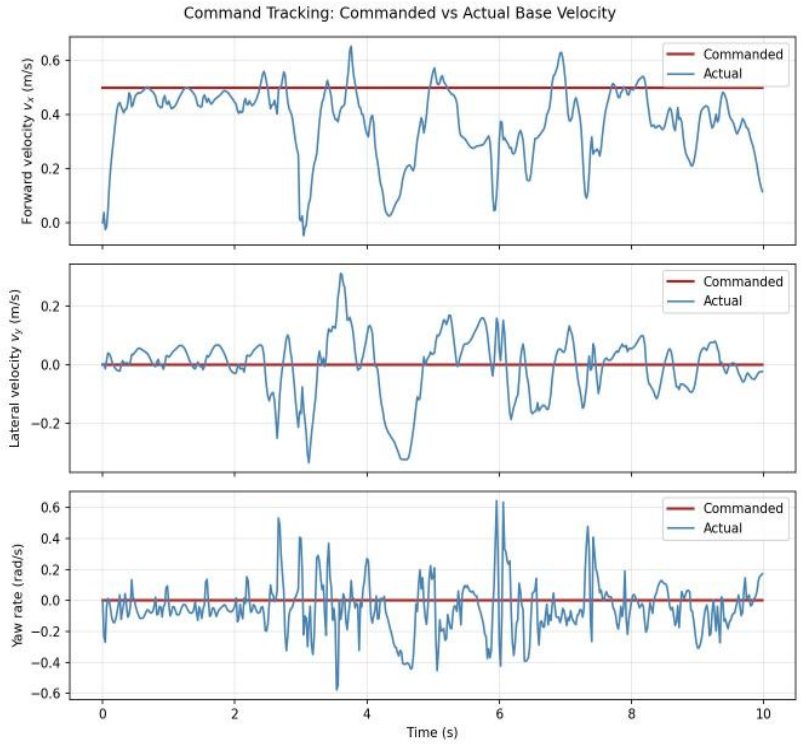


Figure 6: Command tracking of the trained DreamWaQ policy during a 10-second evaluation rollout. The actual body velocity (blue) oscillates around the commanded velocity (red) within each gait cycle.

The same rollout is shown in task space in Figure 7, which plots the robot's base position from above. Starting from the green marker, the base advances steadily in the commanded forward (+x) direction with only minor lateral drift. Together, Figure 6 (task-space velocity) and Figure 7 (task-space position) document the policy's behaviour in the world frame.

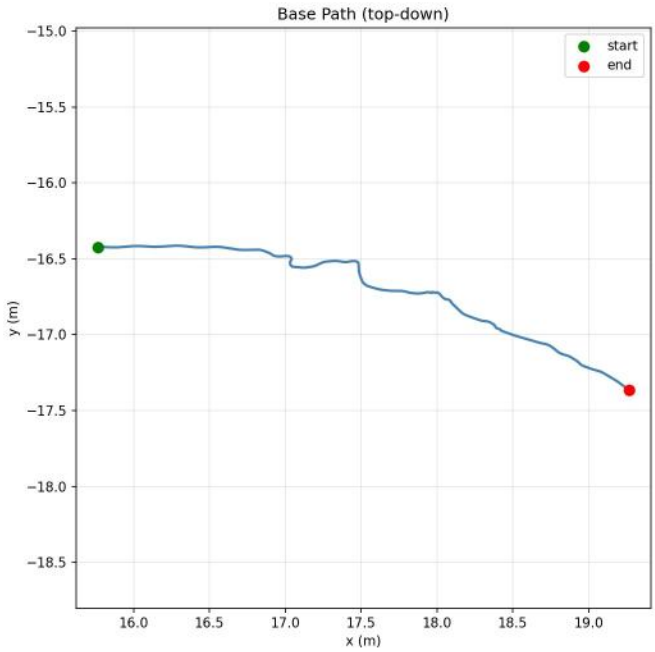


Figure 7: Base path of the trained DreamWaQ policy over the 10-second rollout, viewed top-down in the world x-y plane (task space). The green marker is the start and the red marker is the end.

Figure 8 shows the same rollout in configuration space: the twelve joint angles over time, grouped by hip, thigh, and calf. Each joint oscillates periodically, the signature of the gait cycle, and the four legs follow the same pattern offset in phase, with the angles staying within the robot's mechanical limits throughout. This is the configuration-space counterpart to the task-space velocity and position above.

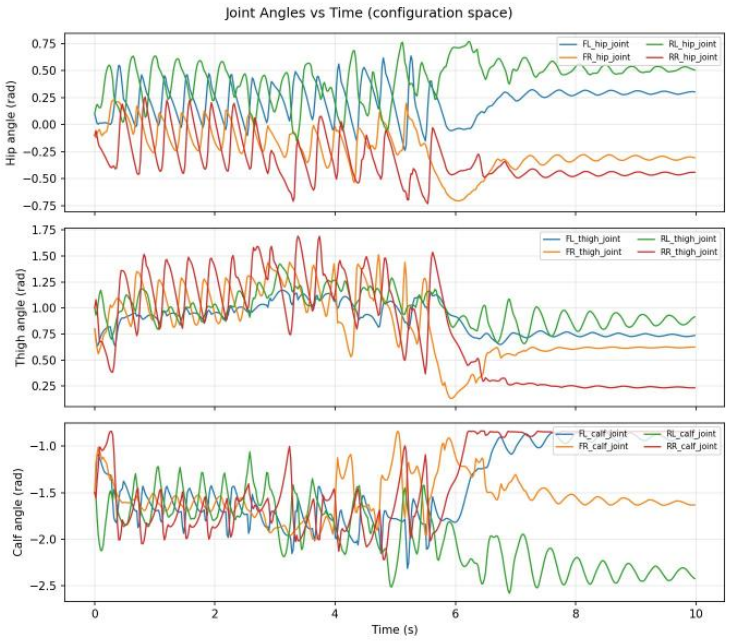


Figure 8: The twelve Go2 joint angles over the 10-second rollout, grouped by hip, thigh, and calf joints across the four legs (configuration space).

4.4 CENet Velocity Estimation

During training, the three components of CENet's hybrid loss were logged each iteration; how they change over training is shown in Figure 9.

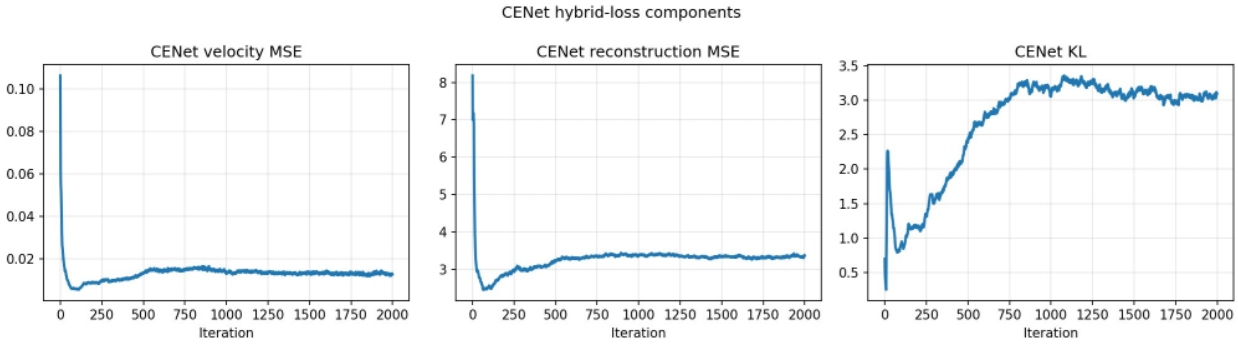


Figure 9: The three CENet hybrid-loss components over training: velocity-estimation MSE (left), reconstruction MSE (middle), and the latent KL term (right).

The velocity-estimation MSE (left panel) drops sharply from about 0.10 to roughly 0.01 within the first hundred iterations, then settles around 0.012 for the remainder of training. This is CENet quickly learning to estimate the robot's body velocity from proprioceptive history alone, the core function of the second contribution. The reconstruction MSE (middle) initially decreases, then rises and settles around 3.3 as the shared encoder balances the competing objectives. The KL term (right) dips briefly, then rises steadily to around 3.0-3.1 by the middle of training and then levels off, meaning the latent stays active and keeps encoding information rather than collapsing to the prior.

Direct estimation accuracy. Beyond the training-loss curve, the quality of CENet's velocity estimate is shown directly in Figure 10, which compares the estimated body velocity against the simulator's ground-truth velocity over an evaluation rollout.

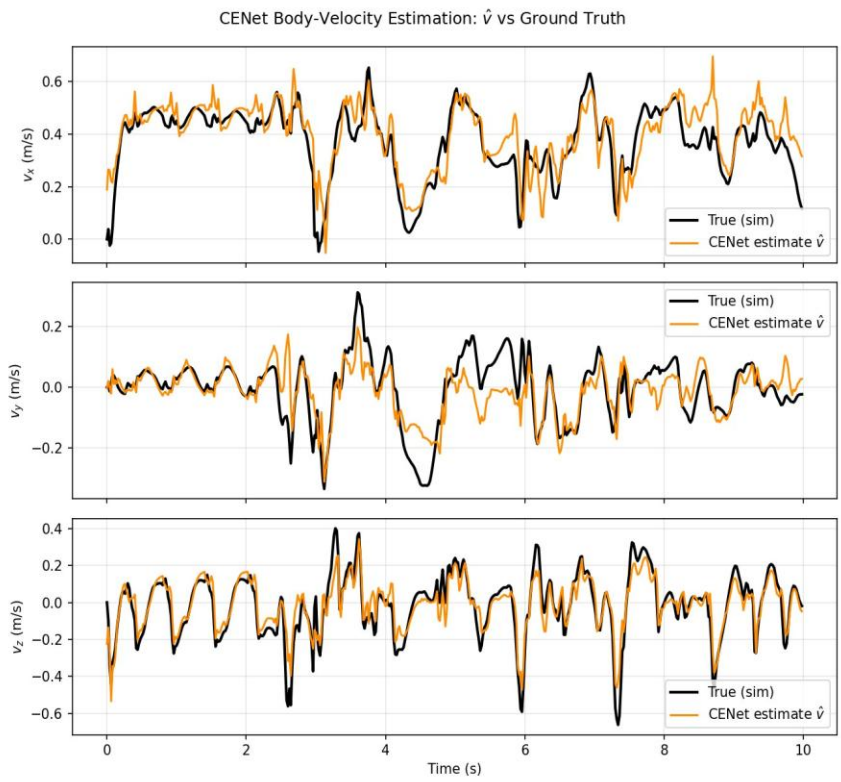


Figure 10: CENet body-velocity estimation accuracy over a 10-second rollout. The estimate (orange) tracks the true body velocity (black) on all three axes, including rapid within-step fluctuations, using proprioceptive history only.

Figure 10 is the main evidence that CENet is accurate. On all three axes the estimated velocity closely follows the ground-truth velocity, including the large vertical (v_z) swings as the body bounces during walking. Across the rollout the velocity-estimation error was a mean-squared error of about 0.0058 $(m/s)^2$. CENet therefore recovers the robot's true body velocity from proprioceptive history alone, the implicit state estimation that makes the asymmetric, proprioception-only policy work.

4.5 Qualitative Results: Trained Robot in Simulation

The trained DreamWaQ policy was visualized by running it in the Isaac Sim viewer across many parallel environments. Figure 11 shows an example frame: Unitree Go2 robots walk over the automatically generated rough terrain, including discrete-obstacle blocks, uneven tiles, and a stepped pyramid section, while following their individual velocity commands (shown as green/blue arrow markers). Several robots are captured walking across the rough, uneven terrain, with the stepped terrain visible behind them. The robots maintain stable, coordinated gaits across the different terrain types, which shows, at least in broad terms, that the proprioception-only policy handles the range of terrain it was trained on. A recording of this rollout accompanies this report as a video file.

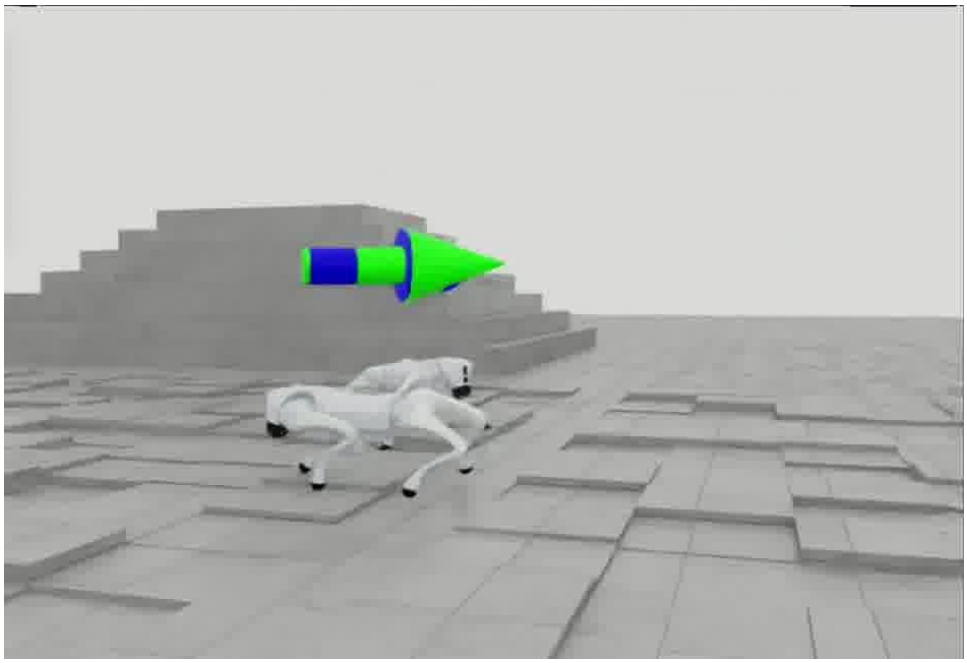


Figure 11: The trained DreamWaQ policy controlling Unitree Go2 robots over rough terrain in Isaac Sim. The robots walk across the uneven, discrete-obstacle terrain, with a stepped pyramid section visible behind them, each tracking its own velocity command (arrow markers).

5. Discussion

5.1 Comparison with the Paper

The central claim of DreamWaQ is that a policy trained with an asymmetric critic and a context estimator can match the performance of a policy with privileged access, while remaining deployable. The results here line up with that claim: the DreamWaQ policy matched a strong privileged baseline on terrain difficulty and episode length while trailing it only slightly on reward (about three) and on tracking (about 0.1 m/s), the expected cost of replacing the baseline's direct access to true velocity and the terrain height scan with CENet's inferred state. CENet's velocity-estimation loss settled at a low value, confirming that the estimator learned to recover body state from proprioception. This reproduction used lighter settings than the original (a Go2 in Isaac Lab rather than the A1 in Isaac Gym, 2048 environments rather than 4096, a single seed, and the simplifications listed in Section 5.2), so the results are best read as a reproduction of the method's core behaviour rather than an exact match of the paper's numbers. Within those limits, the implementation does show the key idea: comparable locomotion from non-privileged observations using a learned context estimator.

Advantages and disadvantages of the method:

On the positive side, DreamWaQ trains in a single phase (no separate teacher and student to train sequentially), needs no external camera or depth sensors at deployment, and converts hard-to-measure body state and terrain into quantities the robot infers for itself, which is what makes the learned policy directly deployable and, in the original paper, robust enough for long outdoor walks. Its main disadvantage is that the adaptation is reactive rather than forward-looking: as the authors themselves note, the robot effectively has to make contact with an

obstacle before it can adapt to it, because the policy has no forward-looking terrain perception. The method also relies on the simulator supplying ground-truth body velocity to supervise CENet, so the quality of the deployed estimator is limited by how closely the simulation matches the real robot and the β-VAE adds a parameter (β) that must be chosen carefully to avoid latent collapse, which appeared at β = 1 (Section 5.2). In short, the method buys deployability and training simplicity at the cost of purely reactive obstacle handling and a dependence on accurate simulation.

5.2 Limitations

- Single training seed, so results are a rough indication, not statistically solid; no error bars across seeds.
- The critic received the privileged terrain height scan and the true base velocity, but not the random disturbance force that the paper also includes in its privileged state; the paper's critic receives the full privileged state, so the critic in this reproduction is slightly less privileged.
- The optional AdaBoot gate was not implemented; the actor used CENet's detached estimates directly.
- The VAE reconstruction target was the current observation rather than the paper's next observation, a deliberate simplification to avoid modifying the rollout storage.
- The latent context was regularized with a β-VAE weight fixed at 0.5, which prevented the posterior collapse seen at β = 1 (where the KL term sits near zero for much of training). This weight was chosen by hand rather than tuned, so a proper sweep or an annealing schedule for β remains worth exploring.
- Training compute was limited to a single 8 GB laptop GPU, constraining run length and the number of seeds feasible within the project timeline, and capping the simulation at 2048 parallel environments rather than the paper's 4096.

5.3 Future Work

- Train multiple random seeds with 4096 environments and report mean and variance, so the baseline-vs-DreamWaQ comparison is statistically supported rather than just a rough indication.
- Sweep or anneal the β-VAE weight (fixed at 0.5 here, which already avoided the collapse seen at β = 1) to study how it trades off reconstruction quality against the information carried by the latent context.
- Implement the AdaBoot adaptive-bootstrapping gate and measure its effect on early-training stability.

6. Conclusion

This project reproduced the core of the DreamWaQ method, an asymmetric actor-critic and the CENet context estimator, in NVIDIA Isaac Lab on the Unitree Go2. The resulting policy approached a strong privileged-observation baseline (matching it on terrain difficulty and stability while trailing only slightly on reward and tracking) using only proprioception, and its estimator recovered the true body velocity to a mean-squared error of about 0.006 $(m/s)^2$ in evaluation, from proprioceptive history alone. Together, these results support the central claim of the paper: a single-phase policy can learn to infer the state it cannot observe and walk robustly on it, with no privileged sensing at deployment.

References

- I. M. A. Nahrendra, B. Yu, H. Myung, “DreamWaQ: Learning Robust Quadrupedal Locomotion With Implicit Terrain Imagination via Deep Reinforcement Learning,” ICRA 2023. arXiv:2301.10602.
- M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg, “Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments,” IEEE Robotics and Automation Letters, vol. 8, no. 6, pp. 3740-3747, 2023. doi:10.1109/LRA.2023.3270034. (The framework used in this work, NVIDIA Isaac Lab, is the successor of Orbit.)
- NVIDIA, “NVIDIA Isaac Sim: Robotics Simulation and Synthetic Data Generation,” software, NVIDIA Corporation. [Online]. Available: https://developer.nvidia.com/isaac/sim
- N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning,” in Proc. 5th Conference on Robot Learning (CoRL), PMLR vol. 164, pp. 91-100, 2022. (The rsl_rl library used for the PPO implementation.)
- Manaro-Alpha, “DreamWaQ (reference implementation),” GitHub repository. Available: https://github.com/Manaro-Alpha/DreamWaQ

Appendix A: Code and Reproduction

Task IDs:

- Baseline: Isaac-Velocity-Rough-Unitree-Go2-v0
- DreamWaQ: Isaac-Velocity-Rough-Go2-DreamWaQ-v0 (and -Play-v0 for evaluation)

Training command (example):

isaaclab.sh -p scripts/reinforcement_learning/rsl_rl/train.py --task Isaac-Velocity-Rough-Go2-DreamWaQ-v0 --headless --max_iterations 2000

Playback / evaluation command (example):

isaaclab.sh -p scripts/reinforcement_learning/rsl_rl/play.py --task Isaac-Velocity-Rough-Go2-DreamWaQ-Play-v0 --num_envs 50

Video of the simulation included

Program files and how to install and run. The code is provided as the self-contained go2_dreamwaq task. To install and run: (1) install NVIDIA Isaac Lab 2.0, Isaac Sim 4.5 following NVIDIA's setup instructions, and install the rsl_rl 3.1.2 library; (2) copy the go2_dreamwaq package into source/isaaclab_tasks/isaaclab_tasks/manager_based/locomotion/velocity/config/ so the task IDs register automatically on import; (3) train the DreamWaQ policy with the training command listed above; (4) visualize or evaluate a trained checkpoint with the playback command listed above; (5) regenerate the figures in this report with the plotting and rollout scripts (plot_results.py for the learning curves and CENet losses, and rollout_eval.py for the command-tracking, base-path, joint-angle (configuration-space), and velocity-estimation plots).

Appendix B: Running the Full Pipeline and Deploying the Policy

This appendix brings the whole project together as a single ordered workflow, from a clean install to a trained and evaluated policy, and then describes how that policy would move from simulation to the physical robot. The individual commands appear in Appendix A; here they are placed in sequence, with the reasoning behind each step, so the work can be reproduced end to end and the path to hardware is explicit.

The pipeline runs in six stages. Each stage depends on a verified result from the one before it, so a mistake is caught early rather than surfacing later as a confusing training curve.

```
# run everything below from the Isaac Lab root, e.g.:  cd ~/IsaacLab
# shortcut to the copied task package:
PKG=source/isaaclab_tasks/isaaclab_tasks/manager_based/locomotion/velocity/config/go2_dreamwaq
```

- **Stage 1 — Install and register.** Install Isaac Lab 2.0 / Isaac Sim 4.5 and rsl_rl 3.1.2, then copy the go2_dreamwaq package into the locomotion config folder. On import, __init__.py registers both task IDs and the custom DreamWaQ classes, so no framework files are edited.

  ```
  cp -r go2_dreamwaq
  source/isaaclab_tasks/isaaclab_tasks/manager_based/locomotion/velocity/config/
  ```

- **Stage 2 Train the baseline.** Train the stock symmetric Go2 task for 2000 iterations at seed 1. This is the privileged reference the DreamWaQ policy is later measured against, and training it first confirms that the environment, reward, and curriculum are healthy before any DreamWaQ-specific change is introduced.

  isaaclab.sh -p scripts/reinforcement_learning/rsl_rl/train.py --task Isaac-Velocity-Rough-Unitree-Go2-v0 --headless --max_iterations 2000 --seed 1

- **Stage 3 Train the DreamWaQ policy.** Train the asymmetric + CENet task with the same 2000 iterations, seed, curriculum, and reward. The only differences from the baseline are the observation routing and the added CENet loss, which is exactly what keeps the comparison fair.

  DREAMWAQ_CENET_BETA=0.5 isaaclab.sh -p scripts/reinforcement_learning/rsl_rl/train.py --task Isaac-Velocity-Rough-Go2-DreamWaQ-v0 --headless --max_iterations 2000 --seed 1

- **Stage 4 Verify the wiring.** Run the offline unit tests (test_cenet.py and test_dreamwaq_actor_critic.py) and check the printed network dimensions (actor 64, critic 235). This catches input-size mistakes, a broken detach, or a non-finite loss before any training result is trusted.

  isaaclab.sh -p $PKG/test_cenet.py
  isaaclab.sh -p $PKG/test_dreamwaq_actor_critic.py

- **Stage 5 Evaluate.** Roll out the trained checkpoint (model_1999.pt) with rollout_eval.py under a fixed velocity command. This produces the command-tracking, base-path, joint-angle, and CENet-versus-true-velocity logs behind Figures 6–8 and 10.

  isaaclab.sh -p $PKG/rollout_eval.py --task Isaac-Velocity-Rough-Go2-DreamWaQ-Play-v0 --num_envs 1 --num_steps 500 --out ~/figures

- **Stage 6 Visualize and plot.** Run the policy in the Isaac Sim viewer for the qualitative figure (Figure 11), and run plot_results.py to regenerate the learning curves and CENet-loss panels (Figures 2–5 and 9) from the TensorBoard logs.

  isaaclab.sh -p scripts/reinforcement_learning/rsl_rl/play.py --task Isaac-Velocity-Rough-Go2-DreamWaQ-Play-v0 --num_envs 50
  isaaclab.sh -p $PKG/plot_results.py --logdir ~/logs/rsl_rl --dreamwaq go2_dreamwaq_rough --baseline unitree_go2_rough --out ./figures

The product of this pipeline is a single checkpoint holding the actor, critic, and CENet weights, together with the figures and logs that document the policy's behaviour. Because the baseline and DreamWaQ runs differ only in those two intended places, any performance gap between them can be attributed to the method rather than to an accident of setup.